Near-optimal broadcasts on arbitrary cactus networks

A polynomial-time approximation scheme for the cactus case posed at ICALP 2025 [1].

The network model

One vertex starts with a message. In each round, every vertex that already knows it can make at most one call along an edge to a neighbor. The objective is to inform all vertices in as few rounds as possible. A cactus is a connected undirected graph whose blocks are bridges or simple cycles. Cycles can be long, and arbitrarily many cycles can meet at a junction.

Theorem

For every fixed ε with $0 < \varepsilon \leq 1$, a schedule can be constructed in polynomial time with the guarantee

$$\text{rounds} \leq (1+\varepsilon)\,\text{OPT}.$$

The running time is $n^{O(1/\varepsilon^2)}$ using polynomial-time rational linear programming.

Why the approximation holds

At a junction, a cycle can be reached by one call around the ring or two calls that split it into paths. Each option contributes at most two deadlines to the same junction. Keep tight deadlines exact. Group later deadlines into buckets and round the choices so each bucket gains at most four calls. A shared time budget then carries the same approximation ratio through every junction.

How this fits the published question

Aminian et al. proved NP-completeness and gave a factor-two approximation for cacti, then asked whether a PTAS exists [1]. Bringolf et al. subsequently obtained a PTAS for cycles sharing one central vertex [2]. This proof covers the full cactus block tree.

A cycle contributes up to two calls

Keep the first call into each vertex

In any schedule, select one call that first informs each vertex other than the source. The sender was informed in a strictly earlier round. Following these parent edges therefore gives a rooted spanning tree. Conversely, a schedule using only a spanning tree is a valid schedule on the graph.

Root the block-cut tree at the source. For each bridge block, the spanning tree must keep its edge. For each cycle block, it must keep a connected acyclic subgraph spanning the block. The outside blocks attach through individual vertices and cannot connect two distinct vertices of this cycle. Thus the spanning tree deletes exactly one cycle edge.

Read a cycle in order around its entry

Write v for the entry vertex and $u_1, \ldots, u_m$ for the other cycle vertices in cyclic order. A cut after u_k leaves a prefix path from u_1 and a suffix path from u_m in reverse order. For k = 0 or k = m, one path is empty. Each nonempty path requires one call from v to its first vertex.

```
Cycle order:       v, u1, u2, u3, u4, u5, v
Cut u2--u3:        v -> u1 -> u2
                    v -> u5 -> u4 -> u3
```

A path state includes the specified path and all descendant blocks hanging from its vertices. At each vertex on the path, a call to the next path vertex is a mandatory job alongside the options for its own child blocks.

Local deadlines

If a branch has a normalized time budget h and v has budget T, its first call has deadline d = T − h. An option for a child block is a vector of one or two such deadlines. Pick one option per block and assign distinct positive call slots. Branches from different chosen calls have disjoint vertex sets, so their schedules can run in parallel.

Choose one plan per block

For any fixed options, unit calls can meet their deadlines if and only if every positive deadline t has at most t calls due by t. Sufficiency follows by sorting the calls by increasing deadline and assigning slots 1, 2, and so on.

Keep the first K deadlines exact

$$\delta = \varepsilon/3, \qquad K = \lceil 8/\delta^2 \rceil, \qquad \rho = (1+\delta)^2.$$

Discard options with a nonpositive deadline. In an exact schedule, at most K blocks select an option containing a deadline at most K, because each such block uses at least one of the first K slots. Enumerate these blocks and fix their chosen options. Reject the guess if its early calls violate any early capacity. Every remaining block is restricted to options with only later deadlines.

Round later deadlines upward

$$b_0 = K, \qquad b_j = \lfloor (1+\delta) b_{j-1} \rfloor \quad (j \geq 1).$$

For a later deadline d, use the first boundary $b_j \geq d$. Then $b_j \leq (1+\delta)d$. Stop the sequence when the largest deadline is covered. Fixed options contribute fixed counts at their early deadlines and rounded later deadlines.

The linear program

For each remaining block i and option o, use $x_{i,o} \geq 0$. Let $A_{j,i,o}$ count its calls due by b_j after bucketing, and let F_j count the fixed calls due by that boundary.

$$\sum_o x_{i,o} = 1 \quad \text{for each } i, \qquad \sum_{i,o} A_{j,i,o} x_{i,o} \leq b_j - F_j \quad \text{for each } j.$$

If the unrounded deadline problem is feasible, the guess matching its early choices passes, and its selected later options give an integral feasible point of this linear program. Thus rejecting every guess rules out exact local feasibility.

At most four extra calls per bucket

Take a feasible rational point. Hold every variable at 0 or 1 fixed. Let m be the number of remaining fractional variables. Keep one equality for the sum of each group that still has fractional variables.

Keep an equality for the current load of a later bucket while its total incidence on fractional variables is greater than four. Incidence counts a variable twice if its option puts two calls in the bucket. This uses individual bucket counts, while the linear program used cumulative counts.

There is a direction to move

Each active group contains at least two fractional variables because its sum is one. Thus there are at most m/2 active group equalities. Each option contributes at most two incidences across all buckets, so the total incidence is at most 2m. A kept bucket has at least five incidences. There are at most 2m/5 bucket equalities.

$$\#\text{equalities} \le m/2 + 2m/5 < m \qquad (m > 0).$$

Find a nonzero direction preserving these equalities. Move in that direction until a variable reaches 0 or 1. Fix that variable. Repeating removes all fractional variables. The group equalities guarantee that exactly one option survives in each block.

Charge only the remaining incidence

Until a bucket equality is dropped, its load is exactly its original fractional load. At the moment it is dropped, at most four incidences can still move. Each remaining variable changes by at most one, so the final bucket load can exceed the original load by at most four. The fractional support only shrinks, so a dropped equality stays dropped.

The fixed early choices were never changed. Consequently the final early calls meet every early deadline, and the number of calls due by b_j is at most $b_j + 4j$.

The slack is charged once

Convert bucket overflow to a call-time bound

For each later bucket, the gap is at least δK/2, since floor(δ b_{j-1}) ≥ δK − 1 and δK ≥ 2. Therefore b_j ≥ jδK/2. The choice of K gives

$$4j \le \frac{8b_j}{\delta K} \le \delta b_j.$$

Sort the selected calls by their rounded deadlines. A call whose original deadline is d in bucket j is assigned a slot at most b_j + 4j ≤ (1+δ)b_j ≤ ρd. An early call is assigned a slot at most d. Thus local acceptance yields a schedule that meets each deadline after stretching it by the same factor ρ.

Store a budget and a schedule for each state

For each rooted state, store the smallest accepted integer budget T and an actual schedule of length at most ρT. A leaf uses budget zero. Inductively, child budgets h are at most the true optimal durations of their states. Hence the true optimal schedule for the parent state chooses options and call slots that meet deadlines formed with these h values. Its duration is accepted, so the stored T is at most the true optimum.

For a chosen branch with budget h, the parent calls it at a slot t ≤ ρ(T−h). The stored branch schedule then needs at most ρh more rounds.

$$t + \text{branch duration} \le \rho(T-h) + \rho h = \rho T.$$

The same inequality applies to each branch and each path continuation. At the source, the stored schedule therefore finishes in at most ρ OPT rounds. With δ = ε/3 and ε ≤ 1, ρ ≤ 1+ε. The inductive guarantee uses one shared factor at every depth.

A polynomial scheme for fixed error

Process the block tree from the leaves

Compute the normalized budget of each vertex with its descendant blocks. For each child cycle, compute every prefix path state in each direction. A path is solved from its far endpoint back toward the entry, using the next path state as one additional mandatory call. The two sets of prefix states supply every cut option. Across the cactus there are $O(n^2)$ such path states.

Bound the work at a junction

There are $O(n)$ available options at any state because a block has one option per cycle edge or one bridge option. Enumerating up to K blocks and their early options takes $n^{O(K)}$ time. Each guess uses a polynomial-size rational linear program. For fixed δ, all deadlines have $O(\log n)$ bits and the bucket sequence has polynomial length.

A feasible rational LP point can be found in polynomial bit time. Each rounding step is a rational linear system with integer coefficients at most two. Clearing denominators gives an integer direction with polynomial bit length by determinant bounds. With a common denominator for the coordinates, one step multiplies that denominator by at most one direction coordinate in absolute value. There are at most as many steps as variables, so all numbers retain polynomial bit length.

Search over budgets

It suffices to scan budgets from zero through $n-1$ because every rooted state can be informed by a spanning tree within $n-1$ rounds. The reference program uses binary search. Acceptance is monotone as deadlines increase. A previously fixed early option that moves entirely past K can be given weight one in the new LP. Every previously fractional option already had only later deadlines and retains that property. All rounded deadlines move weakly later.

The theoretical time bound is $n^{O(1/\varepsilon^2)}$. The runnable reference uses exact rational simplex for its LP calls. The time bound for the scheme uses a polynomial-time LP algorithm.

Schedules and exact comparisons

The replay compared 200 generated cacti with an exact spanning-tree oracle that tries every cycle cut. An independent search over all possible informed sets cross-checked 80 of those instances. The returned schedule reached the exact optimum in all 200 generated cases.

Two larger networks

Network	Vertices	Optimum	Schedule	Budget
Four-cycle hub	134	109	109	97
Cycles on a spine with a star branch	187	91	92	86

Both larger runs used $\varepsilon = 1$ and exercised the late-bucket LP. The nominal budget is a lower bound on the optimum. A separate replay validates every call against the graph and checks that a vertex has received the message in an earlier round before it can send.

A forced fractional local choice

A local instance has four flexible blocks and five later buckets. After subtracting fixed calls, the prefix capacities are 1, 3, 4, 6, 7. All 16 integral choices violate a capacity, while the exact linear program is feasible. The implementation rounds that point in 4 moves and misses the most demanding unrounded deadline by 1 slot.

Source and proof checks

The bundle contains the exact-rational Python scheduler, the test inputs and replay, and Lean checks of the dimension count, the deadline inequality and the composition inequality. The full graph reduction and rounding proof are given in this note.

[1] A. Aminian, S. Kamali, S.-M. Seyed-Javadi, and Sumedha. *On the Complexity of Telephone Broadcasting from Cacti to Bounded Pathwidth Graphs*. ICALP 2025, §6. DOI.

[2] J. Bringolf, H. A. Harutyunyan, S. Kamali, and S.-M. Seyed-Javadi. *On Hardness and Approximation of Broadcasting in Sparse Graphs*. arXiv:2510.20026.